

LAPQ lib

Abstract

This report documents lapq, an experimental C99 skip-list priority queue with learning-augmented insertion hints, Python bindings, graph-based dataset generation, and a roadmap toward machine-learning predictors.

Contents

Abstract	1
1 lapq: Learning-Augmented Priority Queue Experiments	3
2 Detailed Outline	4
3 1. Introduction and Motivation	5
4 2. Theoretical Background	6
5 3. C Library Architecture	7
6 4. Prediction Hints in the C Core	8
7 5. Difference from the Paper	9
8 6. Testing Strategy	10
9 7. C Benchmarks	11
10 8. Python Binding	12
11 9. Graph Experiment Layer	13
12 10. Experimental Methodology and Metrics	14
13 11. Current Road-Graph Results	15
14 12. Machine Learning Roadmap	16
15 13. Algorithmic Roadmap	17
16 14. Packaging and Reproducibility	18
17 15. Current Status	19
18 16. Conclusion	20
19 17. References	21

1 lapq: Learning-Augmented Priority Queue Experiments

This notebook is the living project report. It documents the C library, the Python experimental layer, the relationship with the reference paper, and the roadmap toward machine-learning predictors. Sections marked as work in progress are intentionally left as scaffolding to be completed as the experiments mature.

2 Detailed Outline

1. Introduction and Motivation
 1. Project context
 2. Learning-augmented priority queues
 3. Why skip lists
 4. Scope of this repository
2. Theoretical Background
 1. Standard priority queues
 2. Skip lists
 3. Learning-augmented algorithms
 4. The LAPQ model from the reference paper
 5. Clean comparisons, dirty comparisons, and predictions
3. C Library Architecture
 1. Public API
 2. Opaque queue object
 3. Caller-owned items
 4. Skip-list backend
 5. Handle table
 6. Statistics and instrumentation
 7. Error handling and invariants
4. Prediction Hints in the C Core
 1. Hint representation
 2. No-hint baseline
 3. Predecessor hints
 4. Invalid, stale, and cross-queue hints
 5. Backward correction
 6. Bottom-up expansion
 7. Top-down refinement
 8. Rank hints and current limitations
5. Difference from the Paper
 1. What is implemented
 2. What is inspired by Algorithm 2
 3. What is not implemented yet
 4. Why heaps are not the chosen target
 5. Current theoretical claims and non-claims
6. Testing Strategy
7. C Benchmarks
8. Python Binding
9. Graph Experiment Layer
10. Experimental Methodology and Metrics
11. Current Road-Graph Results
12. Machine Learning Roadmap
13. Algorithmic Roadmap
14. Packaging and Reproducibility
15. Current Status
16. Conclusion
17. References

3 1. Introduction and Motivation

`lapq` is an experimental project with two deliberately separated goals. First, it implements a compact C99 priority queue based on skip lists and prediction hints. Second, it provides a Python layer for datasets, graph experiments, synthetic predictors, and eventually machine-learning models.

Predictions never replace correctness-critical comparisons. They are passed to the C core as hints, and the C core validates and corrects them using the ordinary comparator.

4 2. Theoretical Background

This section will summarize the required background: priority queues, skip lists, learning-augmented algorithms, and the model described in the learning-augmented priority queue paper [BC24].

Work in progress. The final report should clearly distinguish clean comparisons, dirty comparisons, pointer predictions, and rank predictions.

5 3. C Library Architecture

The public API lives in `include/lapq/lapq.h` and keeps `struct lapq` opaque. The implementation is split into focused modules: `src/lapq.c` for the public facade, `src/skiplist.c` for skip-list logic and hinted search, `src/handles.c` for generational handles, `src/stats.c` for instrumentation, and `src/lapq_internal.h` for private shared structures.

The C core stores caller-owned `void *` items. It does not know how predictions are computed and does not own any machine-learning logic.

6 4. Prediction Hints in the C Core

The learning-augmented interface is `struct lapq_hint`. The current C core supports no hint, predecessor hints, and experimental rank hints. Predecessor hints are validated through generational handles. If valid, the search starts near the predicted predecessor; if invalid, the implementation falls back to ordinary search and increments `invalid_hints`.

The hinted search has three explicit correction phases: backward correction, bottom-up expansion, and top-down refinement. These phases are also reflected in the instrumentation counters.

7 5. Difference from the Paper

The implementation is inspired by the pointer-prediction search model, but it does not yet implement the full theoretical system from the reference paper. In particular, dirty comparisons are not present, rank hints are not backed by an asymptotically strong indexed structure, and the repository currently has one concrete skip-list backend.

This section will be expanded with precise claims and non-claims once the experimental results are stable.

8 6. Testing Strategy

The C tests cover insertion and extraction order, duplicate keys, handles, `decrease_key`, arbitrary removal, invalid hints, stale handles, hinted search phases, and a randomized oracle. Sanitizer builds are run through `make check`.

The Python tests cover the CPython binding, handle-returning insertions, hint passing, graph loading, Dijkstra baselines, dataset generation, and CLI paths.

9 7. C Benchmarks

The C benchmark compares baseline insertions with perfect, noisy, and bad predecessor hints, plus a dedicated `decrease_key` workload. It reports time, clean comparisons, level-0 steps, express-level steps, per-phase traversal counters, hint counts, invalid hints, average error, maximum error, and a checksum.

Work in progress. This section should eventually include tables and plots from reproducible benchmark runs.

10 8. Python Binding

The Python package exposes `PriorityQueue` and `Handle` through a CPython extension. The binding maps insertion, extraction, peeking, removal, stats, invariant checks, and predecessor/rank hint insertion to the C core.

Python-facing `decrease_key` is intentionally not exposed yet. The binding needs an explicit policy for mutating the priority stored inside C-owned queue items before calling the C `lapq_decrease_key` operation.

11 9. Graph Experiment Layer

The graph layer loads DIMACS shortest-path graphs into a compact CSR representation. It provides Dijkstra baselines with Python `heapq`, Dijkstra with `lapq`, and Dijkstra with synthetic LAPQ hints. Dataset utilities emit CSV rows for priority-queue insertion events, including Python-computed oracle predecessor information.

The current dataset workflow is intentionally simple and reproducible. A road graph is loaded once, a fixed set of pseudo-random sources is selected from a seed, Dijkstra is run once per source, and at most a fixed number of queue-insertion events is emitted per run. The current New York dataset snapshot was generated from `USA-road-d.NY.gr` with eight sources, seed 123, and at most 50,000 events per source.

The generated queue-insertion CSV stores only data that is useful for the current experiments:

- `run`, `step`, `source`, `node`, `target` identify the event and the Dijkstra run;
- `distance`, `edge_weight`, `queue_size_before` describe the insertion context;
- `predecessor_rank`, `predecessor_distance`, `predecessor_node`, `predecessor_sequence` record the oracle predecessor for replay and future supervised learning.

The road graph files under `graphs/dimacs/` are local data and are intentionally kept out of Git.

12 10. Experimental Methodology and Metrics

The main algorithmic metric is the number of **clean comparisons** performed by the C priority queue. Wall-clock time is still recorded, but it is treated as an engineering metric rather than the primary scientific signal.

This choice is important because the Python experiments include costs that are not part of the C data structure itself: CPython call overhead, Python handle management, synthetic hint generation, and auxiliary oracle structures. These costs can hide the algorithmic effect of predictions. Clean comparisons instead directly measure how much correctness-critical work remains after a hint is supplied and corrected.

The current Python pipeline uses three complementary experiment types:

1. **Dijkstra backend comparison.** Runs shortest paths with `heapq` and with `lapq` without hints.
2. **Dijkstra synthetic hints.** Runs Dijkstra with LAPQ and synthetic hint policies while preserving the full Dijkstra workflow.
3. **Replay benchmark.** Replays a saved stream of priority-queue insertions after precomputing hint plans. This removes most graph/oracle overhead and focuses on how the queue behaves when hints are already available.

CSV reports are organized so that comparison counts, comparison ratios, and comparison reductions appear before timing columns.

13 11. Current Road-Graph Results

The current road-graph snapshot uses the DIMACS New York graph with eight pseudo-random sources. The generated queue-insertion dataset contains 400,000 events, split evenly across eight Dijkstra runs.

The dataset-level summary shows that each run contributes 50,000 events. The average active queue size is in the hundreds, which makes predecessor prediction nontrivial while keeping replay experiments manageable.

The most important current result comes from replaying the saved insertion stream and measuring clean comparisons relative to the LAPQ baseline:

scenario	rows	clean com- parisons	comparison ratio	comparison reduction	average error	max error
baseline	400,000	12,383,515	1.000	0.00%	0.000	0
perfect	400,000	2,391,779	0.193	80.69%	0.000	0
noisy	400,000	8,546,554	0.690	30.98%	63.982	64
bad_left	400,000	20,922,309	1.690	-68.95%	128,375.903	399,985

This validates the intended learning-augmented behavior: accurate predecessor hints substantially reduce clean comparisons, moderately noisy hints can still help, and adversarially bad hints increase work while preserving correctness.

The Dijkstra-level synthetic hint experiment tells a complementary story. Perfect and rank hints reduce clean comparisons by 63.68% relative to the no-hint LAPQ baseline on the single-source NY run, while the current noisy and bad-left policies increase comparisons in that full Dijkstra setting. These timing numbers should not be over-interpreted, because the Python oracle and hint-generation path are still part of the measured end-to-end runtime.

14 12. Machine Learning Roadmap

Real ML models are intentionally out of scope until the dataset format, replay workflow, and baseline measurements are stable. The current repository now has the required scaffolding: DIMACS loading, CSR graphs, multi-source dataset generation, oracle predecessor extraction, comparison-first analysis, and replay.

The first supervised target should be a predecessor-oriented quantity rather than a raw C handle. Candidate targets include:

- predecessor rank/order in the active priority queue;
- an approximate predecessor bucket;
- a hybrid policy that decides when to provide no hint.

Candidate features may come from Dijkstra state, graph topology, current distance values, queue size, edge weights, source/target identifiers, and normalized event position. The first models should be deliberately simple: heuristic baselines, linear or tree-based regressors/classifiers, and only then heavier ML methods if the simpler models justify them.

15 13. Algorithmic Roadmap

The C core is complete for v0.1, but not complete with respect to all ideas in the reference paper. Future algorithmic work is deliberately staged:

- dirty comparisons;
- stronger rank hints;
- indexed skip lists or vEB-like auxiliary structures;
- backend abstraction once multiple concrete backends exist.

These are roadmap items, not current bugs. The criteria for changing the C core should come from measured Python experiments and clear theoretical goals.

16 14. Packaging and Reproducibility

The project uses `setuptools` and a CPython extension. GitHub Actions runs tests and builds wheels for Linux, macOS, and Windows. Doxygen API documentation can be generated locally with `make api-docs`; the generated HTML is ignored by Git.

Release artifacts are built from Git tags and attached to GitHub Releases.

17 15. Current Status

Completed components include the C skip-list priority queue, handles, predecessor hints, experimental rank hints, instrumentation, C benchmarks, Python bindings, DIMACS/CSR graph loading, Dijkstra baselines, synthetic hint scenarios, multi-source dataset CSV generation, comparison-first analysis, replay benchmarks, packaging, CI, and Doxygen infrastructure.

Current limitations include the absence of dirty comparisons, strong rank-aware structures, Python-facing `decrease_key`, and real machine-learning models.

The current empirical story is strong enough to guide the next phase: on the NY replay dataset, perfect predecessor hints reduce clean comparisons by 80.69%, noisy hints with bounded error still reduce them by 30.98%, and bad hints increase work without compromising correctness.

18 16. Conclusion

The repository currently provides a stable experimental foundation: a compact C core with clean correctness semantics and a Python layer designed to generate, replay, and evaluate predictions. The experiments now use clean comparisons as the primary algorithmic metric, which makes the effect of prediction quality visible despite Python-level overheads.

The next research step is controlled feature extraction and simple supervised prediction. The goal is not to introduce ML for its own sake, but to test whether learned predecessor hints can reproduce some of the comparison savings observed with oracle and synthetic hints.

19 17. References

Bibliography

- [BC24] Ziyad Benomar and Christian Coester. Learning-augmented priority queues. In *Advances in Neural Information Processing Systems 38*, 2024. Local copy: [docs/papers/learning_augmented_priority_queues.pdf](#).